B2.1 Programming fundamentals (part 1)

How can we apply programming to solve problems?

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- B2.1.1 Construct and trace programs using a range of global and local variables of various data types
- B2.1.2 Construct programs that can extract and manipulate substrings

B2.1.1 Variables

- **Variable:** a designated memory location that stores a value that can change during the execution of a program.
- **Loop / iteration:** a repetition.
- **Selection:** a conditional statement or decision statement, e.g. IF, CASE statements.
- **Data storage:** storage of data within primary or secondary memory.
- **Operator:** a character that represents a mathematical, arithmetic or logical operation.
- **Identifier:** a lexical token that names the language's entities.
- **Declaration:** a language construct specifying the properties of an identifier.
- **Initialization:** assigning an initial value to a data structure.

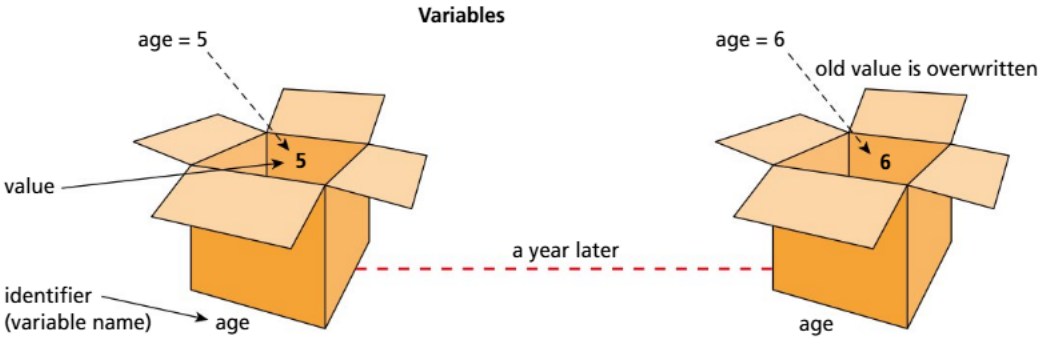
Converting an algorithm into code involves using **variables** to store and manipulate data, **loops** to repeat instructions, and **selection** structures to make decisions on a path to follow to complete a task. Important constructs to understand when developing a program are:

- **data storage**: the use of variables and constants
- **operators**: used to manipulate and compare data (mathematical and logical operators)
- selection/branching structures: used to construct decision statements
- iteration: loops to repeat blocks of code: counter-based and conditional looping structures.

Data storage – use of variables

Consider a sales representative receiving a fixed-base salary, supplemented by a bonus that is tied to monthly sales performance. This bonus fluctuates from month to month; hence it can be characterized as variable over time. In fields like Mathematics and Computer Science, the term "variable" is used to encapsulate such dynamic values.

A variable has an **identifier** (name) and a current value. Each variable can only hold one value at a time. Before being used, a variable must be declared and initialized.



Variable **declaration** refers to specifying the data type of the variable, while **initialization** refers to providing it with an initial value.

◆ **Comment:** a note that explains some code, which will be ignored at compilation stage.

Top tip!

In Python, there is no need to declare the variables used. Therefore, for assessment purposes, they can be mentioned via a **comment** (a comment is used to provide explanations of code, or notes, to the developer, but it is removed at lexical analysis stage – it is not necessary during the compilation of the program, so it will be ignored).

ACTIVITY

Inquirer: Nurture your curiosity, developing skills for inquiry and research.

Research skills: Research the term "constant" and understand the difference between constants and variables. Outline those differences and specify when each of them could be used.

Data types

◆ **Data type:** defines the type of value a variable or data structure has, and what type of mathematical, relational or logical operations can be applied without causing an error.

◆ **String:** a data type used to represent a sequence of characters, digits and / or symbols.

◆ **Assignment:** to set, reset or copy a value into a variable.

◆ **Integer:** a data type used to represent a whole number.

◆ **Float:** a data type used to represent a decimal number.

◆ **Double:** a data type used to represent a decimal number.

The **data type** tells you what type of value a variable will store and what kind of operations are allowed on that specific value.

Is the variable going to store a whole number or a decimal number; is it a piece of text or just a true / false value; or one single character? Every programming language has its own way of declaring variables.

String

String is used to store a sequence of characters, digits and / or symbols (a text). The text is written in double quotation marks in Java, while Python can use single or double quotation marks.

Java

```
String password="Bob@123";
```

Python

```
password ='Bob@123'
password ="Bob@123"
```

In this example, the password is **assigned** (becomes) the value Bob@123.

The primitive data types considered for the curriculum are: `int`, `double`, `char` and `Boolean`.

Integer

Integer (int) is used to store whole numbers (positive or negative integers).

Java

```
int age=54;
```

Python

```
age = 54
```

Decimal

The **float** and **double** data types are used to store decimal numbers (double precision). As double has a higher precision than float, it is safer to use double in your exercises.

Java

```
double salary=5998.96;
```

Python

```
elevation = -3.1
```

◆ **Char:** a data type used to represent one single character, digit or symbol.

◆ **Boolean:** a data type to represent one of the two possible values: true or false.

Char

Char is used to store a single character, digit or symbol.

Java

```
char at= '@';
```

Python

```
at = '@'
```

Boolean

Boolean is used to store one of the two possible values: true or false. So, a Boolean variable could be used to store such data as whether or not a product is still in stock; whether or not a person is a male; whether or not a trip has been paid for, and so on.

Java

```
boolean a = false;
```

Python

```
a = False
```

Boolean variables are often used to evaluate logic expressions. In code, conditions often need to be added and, if the condition would evaluate to true, some statements would be executed; otherwise, different statements would be executed.

Another example of the need for a Boolean variable would be to continue repeating a piece of code as long as an expression evaluates to true or false, based on the requirements.

Consider the following variables: `a = 7` and `b = 54`.

`((a<9) and (b>30))` evaluates to true: if both conditions evaluate to true, the result is true. 7 is smaller than 9 and 54 is greater than 30 (both conditions are met).

`((a>3) or (b<3))` evaluates to true: if either condition is true, the result is true. (The first condition is true; the second is false.)

REVIEW QUESTIONS

1. Define the term "variable".
2. Explain why variables are used in programming.
3. State three data types used in programming.
4. Suggest a way to declare a variable in the programming language you are currently studying.
5. Identify rules and conventions that you could follow when naming variables.
6. Explain why it is important to choose an appropriate data type for a variable.
7. Identify a situation where you need to change the data type of a variable during the execution of a program.
8. Identify an example of a common error when using variables and explain how you would fix it.
9. Explain how the choice of data type affects memory usage and performance in a program.

10 Evaluate the following Boolean expressions if `a = 8` and `b = 3`:
   a E = (a<b) or (a>5)
   b E =! (a>=b)
   c E = (a<8) and (b>3)
   d E = (a==8)
   e E =! (a==b) or (a>b)
11 Identify the most appropriate data type to store:
   a your name
   b your age
   c your phone number
   d whether an item is out of stock
   e the price of a flight ticket.
12 Identify three legal and three illegal identifier names in the programming language you study.

PROGRAMMING EXERCISES

1 Construct code to output a joke on the screen.
2 Construct code to ask the user to enter their name, store it in a variable and display it on the screen, together with a welcome message.
3 Copy the following expressions and display the value of E after each one of them. Check whether your answers to review question 10 above are correct.

```
a = 8
b = 3
E = (a<b) or (a>5)
E =! (a>=b)
E = (a<8) and (b>3)
E = (a==8)
E =! (a==b) or (a>b)
```

ACTIVITY

Use your answers to the programming exercises above to answer the following questions.
1 Did you follow variable naming conventions to solve questions 1 and 2?
2 Were the variable names meaningful and descriptive?

ACTIVITY

Self-management skills: Set goals that are challenging and realistic: Practise five coding challenges of your choice per week. This will greatly improve your coding skills, and it will increase your self-confidence.

ACTIVITY

Communicators: Express yourself confidently and creatively in many ways. Collaborate effectively with and listen carefully to the perspectives of other class members.

Communication skills: Use appropriate forms of writing for different purposes and audiences. Explore and create a table to present to the class the different ranges available for the data types you have studied.

Assignments

Assignment refers to setting a value to a variable; this operation is typically carried out using the equals sign (=). The value on the right of the equals sign is assigned to the variable on the left side of the equals sign; it can never be done the other way around.

```
count = 1
```

This statement assigns the value of 1 to the variable `count`. In other words, `count` is now 1.

But you will often see statements like this: `count = count + 1`. This statement means that the variable `count` is incremented (or increased) by one, or its new value is one greater than it was. **Incrementing** a variable by one is a special case, and you can also write it as `count++`. If ++ means the variable is incremented by one, **decrementing** a variable by one becomes `count--` or `count = count - 1`.

- **Increment:** to increase a value by another value (usually by one).
- **Decrement:** to decrease a value by another value (usually by one).

Another example is when you decrease the variable by a value other than one, such as:

```
price = 5000
price = price - 100
```

Here, the variable `price` becomes 100 lower than it was. So, it was initially 5000, and after the second line of code is executed the new price is 4900. When assigning new values to variables, the previous value is overwritten, so the variable occupies the same memory location. Therefore, in this case, after the two lines of code are executed, the value of 5000 is completely lost.

As such, a challenging question would be: how do you swap the contents of two variables? Imagine that you have two variables, `a` and `b`, storing the values 5 and 7 in this exact order. How could you swap their contents, and end up with `a` storing the value of 7 and `b` storing the value of 5?

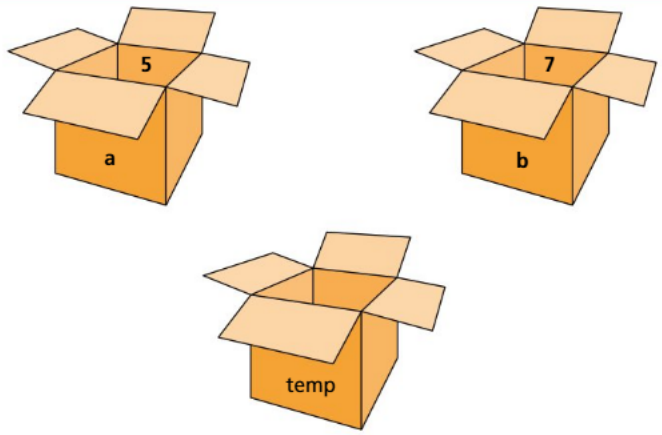
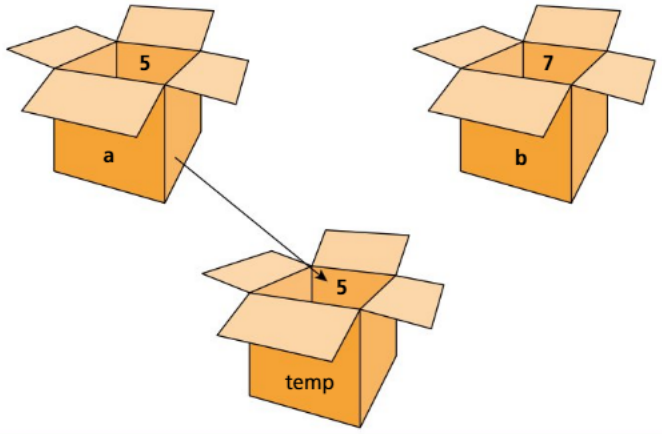
One attempt to solve the problem might be the following:

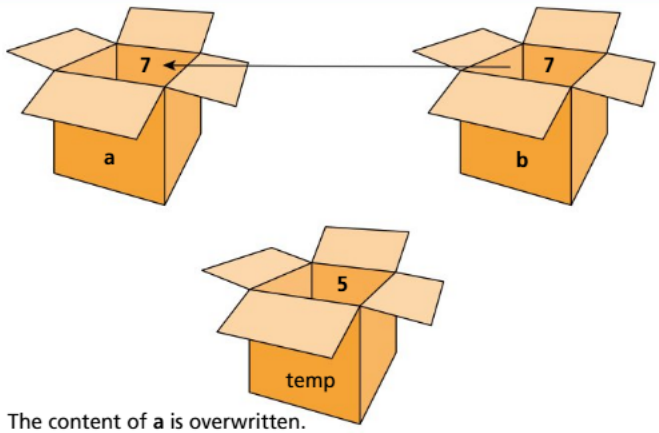
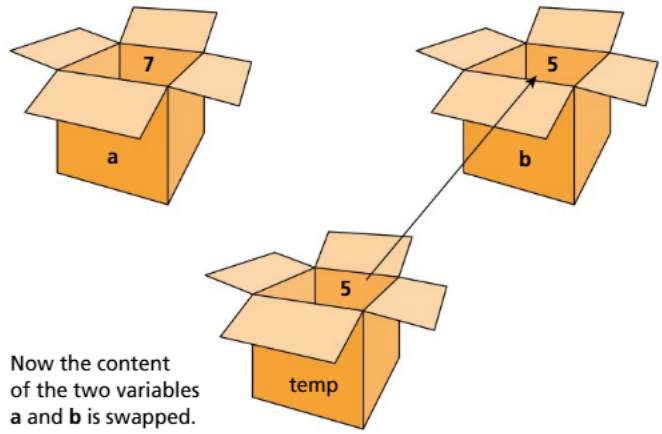
```
a = 5
b = 7
a = b
b = a
```

If you have been tempted to do this, what happens is that you end up with two variables storing the same value; in this case, 7. On line 3, the variable `a` becomes 7, and on line 4, the variable `b` becomes `a`, which means `b` becomes 7 as well.

Therefore, to solve such a problem, you need to imagine that, instead of numbers, you are dealing with liquids. Imagine that the variable `a` is a cup that is filled with water, and the variable `b` is a cup filled with tea. What you want is to swap the contents of your cups: the water to get into cup `b` and the tea into cup `a`. You cannot mix those contents, so what is the solution? A third cup! The solution is to bring in a third cup, which will temporarily hold the content of one of your cups. So, you pour the water into cup `c`. Cup `a` is now available to store the content of cup `b`, which is the tea. After this step, you can pour the water from cup `c` into cup `b`. By doing this, your contents are swapped successfully. The example below shows you how this works with numbers:

```
a = 5
b = 7
temp = a     // 5 is saved into the temporary variable temp
a = b        // 7 is stored into a
b = temp     // 5 (from temporary variable) is stored into b
```

Initial state of the variables.	
Step 1: content of **a** is copied into temp.	

Step 2: content of the variable **b** is copied into **a**.	 The content of **a** is overwritten.
Step 3: content of the variable temp is copied into **b**.	 Now the content of the two variables **a** and **b** is swapped.

Although this might seem an irrelevant challenge right now, this swapping method is part of several sorting routines that you will study later.

Operators

Operators are used to perform calculations, comparisons and other logical operations. Operators can be **arithmetic operators**, such as `+`, `-`, `/`, `*`, `%`; or **Boolean operators**, such as `!`, `&&`, `||`; or **relational operators**, such as `<=`, `<`, `>`, `>=`, `==`, `!=`.

◆ **Arithmetic operator:** a character that is used to perform a calculation.

◆ **Boolean operator:** a character that represents a specific logical operation that is used to produce a true or false outcome.

◆ **Relational operator:** an operator used to compare values or expressions.

Operator in Java	Operator in Python	Meaning
`+`	`+`	addition
`-`	`-`	subtraction
`*`	`*`	multiplication
`/`	`/`	division
`%`	`%`	modulus (returns the remainder)
`<`	`<`	smaller than
`<=`	`<=`	smaller than or equals to
`>`	`>`	greater than
`>=`	`>=`	greater than or equals to
`==`	`==`	equals to
`!=`	`!=`	not equals to
`&&`	`and`	and
`||`	`or`	or

◆ **Binary operator:** an operator that requires two operands (values).

◆ **Operand:** a value used in a mathematical expression.

◆ **Unary operator:** an operator that requires one single operand.

◆ **Integer division:** division in which the fractional part is discarded.

◆ **Floating-point division:** division in which the fractional part is kept.

Arithmetic operators are used to perform calculations such as addition, multiplication, subtraction, and so on. The arithmetic operators presented so far are **binary operators**, meaning they require two **operands** (two values) to apply the calculation on. There are also **unary operators** (that require only one operand), such as:

Unary operator	Meaning
-	negative numbers
++	incrementing the value by 1
--	decrementing the value by 1

While it is quite straightforward to understand when you would use the addition, subtraction or multiplication operators, it might be a bit trickier to understand what the *div* and *mod* operators are.

Div: division operator

In Java and Python, there are two types of division: **integer division** and **floating-point division**. Both types use the same symbol (forward slash) in Java. However, when dividing two integer values, the result will be an integer (integer division); when dividing two floating-point number numbers or a decimal and an integer, the result will be a decimal number (floating-point division).

Java

```
int no1 = 7;
int no2 = 2;
System.out.println(no1/no2);
```

In the example above, even if the result would be 3.5, the answer displayed would be 3, as the two numbers are whole numbers (integers).

Java

```
double no1 = 7.0;
double no2 = 2.0;
System.out.println(no1/no2);
```

However, in this example, as both variables store decimal numbers, the result displayed is a decimal number as well (3.5).

Java

```
float no1 = 7.0;
int no2 = 2;
System.out.println(no1/no2);
```

In the example above, the values stored are numbers of mixed data types, and the result will be a decimal number: 3.5.

In Python, as the type of variables is not specified, there are different operators to represent the different types of divisions. Floating-point division is performed by the / (forward slash) operator, so the result will be a decimal number. However, integer division uses the // (double slash) operator. // will return the floor division (this means that, no matter the result, it will always round it down – what happens is that the decimal part is truncated or, in simpler words, it is ignored or deleted.

Python

```
no1 = 7
#no1 = 7.0
no2 = 2
print(no1/no2)
```

In this example, the output will be 3.5, no matter the data type of the two numbers.

Python

```
no1 = 7
#no1 = 7.0
no2 = 2
print(no1//no2)
```

However, this time the output will be 3, as the result is truncated, without taking into consideration the data type of the variables.

PROGRAMMING EXERCISES

Construct code in the language of your choice to solve the following problems.

1. Ask the user to enter three numbers. Output their average. For example, if the input is: 3, 4 and 5, the output is 4.
2. Ask the user to enter their name and age. Output a message that includes the name and the age that the user will be in 10 years. For example, if the input is Bob, 15, the output should be [Bob, in ten years you'll be 25 years old].
3. Ask the user to enter a three-digit number. Output the sum of all three digits. For example, if the input is 125, the output should be 8.

Common mistake

Algorithms written to solve a problem need to be specific and accurate. Many students lose marks for missing small details, like forgetting to initialize a variable such as a counter or a total.

ACTIVITY

Use your solutions to the programming exercises above to answer the following questions.

1. Did the correct mathematical operations occur for question 1?
2. How did you concatenate the name and the age to display the output for question 2?
3. Was the expected result displayed for question 3?

ACTIVITY

Communication skills: Give and receive meaningful feedback – work in pairs to exchange solutions to the programming exercises and give each other feedback on what could have been done to improve or optimize the proposed solutions.

B2.1.2 String manipulation

In coding, there is often a need to manipulate text. You might want to display some special characters, such as double quotations `""`, single quotations `''` or a backslash `\`.

As those characters are already used for a specific purpose in most programming languages, displaying them might be challenging. At the same time, programmers might want to extract parts of text belonging to a string, join them together, alter or delete them. How is all this possible?

Including an escape character (backslash) supports typing special characters that are usually used for specific purposes in the language. For example, single quotations are used for storing a character in Java or even a string in Python; the same happens with double quotations. Therefore, when wanting to include single or double quotations in the text, you must use the backslash:

Character	Java and Python
"	\"
'	\'
\	\\

Text blocks

In Java, multiple line strings can be written like this:

Java

```
System.out.println("Write multiple\n"
+ "Lines like this");
```

Text on different lines is joined together via the + operator. `\n` represents the new line character, denoting that the new line of text will be displayed on the next line.

In Python, multiple line strings can be written by using triple double quotation marks:

Python

```
print(""" Write multiple
Lines like this
""")
```

The programming language offers several built-in functions that can be used to manipulate strings.

In the examples below, `text` is a variable that stores a piece of text, such as: "Computer Science is fun!"

Length

The length function returns the length (number of characters, spaces included) of the value stored in the string `text`. In this situation, the value stored in `x` is 24.

Java

```
x = text.length()
```

Python

```
x = len(text)
```

Concatenation

◆ **Concatenation:** joining strings together.

Concatenation refers to joining two or more string values together.

Both Java and Python allow several ways to achieve concatenation. One of them is with the + operator, which will join the two strings together.

Java

```
String part1 = "Computer Science is fun";
String part2 = ", isn't it?";
String text = part1 + part2;
System.out.println(text);
```

Python

```
part1 = "Computer Science is fun"
part2 = ", isn't it?"
text = part1 + part2
print(text)
```

Another function that can be used in Java to concatenate two strings is the function `concat`:

Java

```
String part1 = "Computer Science is fun";
String part2 = ", isn't it?";
String text = part1.concat(part2);
System.out.println(text);
```

Note that concatenation is a technique that is applied to a series of string variables, rather than a combination of strings and integers or decimals. If there is a need to concatenate a combination of strings and integers, the + operator can be used in Java, or the integer or decimal value can be converted to a string prior to the concatenation taking place. In Python, the interpolation operator (`%`), the `str` function, `str.format` or `f`-strings can be used for this purpose.

Java

```
String exam = "Computer Science";
int grade = 9;
System.out.println("Your "+ exam + " exam score is " + grade);
```

Python: Use of interpolation operator

```
exam = "Computer Science"
grade = 9
print("%s%s%s%s" % ("Your " , exam, " exam score is ", grade))
```

Python: Use of str function

```
exam = "Computer Science"
grade = 9
print("Your " + exam + " exam score is " + str(grade))
```

Python: Use of str.format

```
exam = "Computer Science"
grade = 9
print("{}{}{}{}".format("Your " , exam, " exam score is ",
grade))
```

Python: Use of f-strings

```
exam = "Computer Science"
grade = 9
print(f'{"Your "}{exam}{" exam score is "}{grade}')
```

Top tip!

In Python, if you want to display the content of the two variables without saving it into another variable, you can simply use the `print` function, which accepts several parameters separated by a comma:

Python

```
part1 = "Computer Science is fun"
part2 = ", isn't it?"
print(part1, part2)
```

Substring

`substring` is the function that is used to retrieve part of the string, for example if you want to extract the first word or letter in a string, or the text between specific positions in the string.

Note that the first position in a string is `0`.

In Java, the function used for this purpose is called `substring`:

Java

```
String text = "Computer Science is fun";
String part = text.substring(8);
System.out.println(part);
```

By providing one argument to the `substring` function, it indicates the starting index of the text to be extracted. In this example, the output would be "Science is fun" as the variable `part` will be assigned the value from the string, starting with position 8 until the end of the string.

Java

```
String text = "Computer Science is fun";
String part = text.substring(8,16);
System.out.println(part);
```

In the example above, the call of the `substring` function is passed two arguments. The first one (8) indicates the starting index (position in string) and the second one (16) indicates the ending index. The substring produced will be from starting index until the ending index –1. Therefore, the text produced in this example will be "Science". This is the case because "S" is the letter at index 8 and "e" is the letter at index 15. The letter at index 16, which is a space character, is not included.

In Python, the `substring` function is often referred to as slicing.

Python

```
text = "Computer Science is fun"
part = text[0:1]
print(part)
```

The code above extracts the first character of the `text` variable: "C".

Python

```
text = "Computer Science is fun"
part = text[:5]
print(part)
```

In this case, the `part` will include the first five characters from the text: "Compu", as "C" is in position 0 and "u" in position 4.

Python

```
text = "Computer Science is fun"
part = text[-1]
print(part)
```

Because the index is –1, this piece of code will store the last character into the string `part`; in this case, the letter "n".

Python

```
text = "Computer Science is fun"
part = text[-6:]
print(part)
```

In this example, the last six characters in the string will be assigned to the variable `part`: "is fun".

Python

```
text = "Computer Science is fun"
part = text[1:-4]
print(part)
```

Above, the extracted text will start at index 1 and will end at the last index –4. So, the value stored in the variable `part` is "omputer Science is".

Replace

The `replace` method searches a string for a character or set of characters and replaces it or them with another character or with other characters.

Java

```
String text = "Computer Science is fun";
text = text.replace('e', '@');
System.out.println(text);
```

In Java, the `replace` method will replace one single character, so the text now becomes "Comput@r Sci@nc@ is fun". To replace several characters, the `replaceAll` method should be used.

Java

```
String text = "Computer Science is fun";
text = text.replaceAll("is", "will be");
System.out.println(text);
```

In Python, the `replace` method is used for replacing both one single character and more characters.

Python

```
text = "Computer Science is fun"
text = text.replace("e", "@")
print(text)
```

Python

```
text = "Computer Science is fun"
text = text.replace("is", "will be")
print(text)
```

Strip

Sometimes, when reading values from a text file or any permanent storage, you might want to remove the trailing white spaces. This can be achieved by using the `strip` method.

Some versions of Java accept `trim` instead of `strip` for the same purpose:

Java

```
String text = "   Computer Science is fun   !   ";
text = text.trim();
System.out.println(text);
```

The leading and trailing spaces will be removed, therefore the new text that will be output is: "Computer Science is fun !"

To achieve the same output in Python, you can use the `strip` function:

Python

```
text = "    Computer Science is fun    !    "
text = text.strip()
print(text)
```

TOK

How does knowledge in Computer Science develop?

Knowledge in Computer Science develops through a dynamic interplay of various Ways of Knowing and Areas of Knowledge. Logical reasoning and empirical evidence form the backbone of technical advancements, while intuition, creativity and ethical considerations shape the broader impact and direction of the field. The interdisciplinary nature of Computer Science ensures that it continually evolves, influenced by and influencing other domains of knowledge. This multifaceted development makes Computer Science a rich field for TOK debates, highlighting the complexity and depth of how knowledge grows and transforms within it.

Is Computer Science knowledge primarily objective, grounded in mathematical truths and empirical data, or does it also encompass subjective elements, such as user experience and ethical considerations? How significant is intuition in developing new algorithms or systems? Can purely logical and empirical approaches lead to all breakthroughs, or is there a place for creative intuition?

PROGRAMMING EXERCISES

1. Construct code that asks the user to provide their name, house / flat number and their street number or name.. Concatenate this information to display a message such as the following. (Attempt to write the message by using one single line of code, and ensure it is displayed on two separate lines, as shown.)

   From: Name

   Address: Full Address

   Message:

   Why was there a bug in the computer?

   Because it was looking for a "byte" to eat!
2. Construct code that allows the user to enter their first and last names. Concatenate the two values, add a space in between and display the full name together with its length without the space.
3. Construct code that allows the user to enter a noun and a letter. Replace all occurrences of that letter with the @ symbol.